

Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introductions to Computer Architecture
In-Class Exercise (10/29)

1. Consider a **64B 2-way set-associative** cache with **16-byte blocks** and **LRU replacement**. Assume memory addresses are 8-bit, and memory is byte-addressable.
 - a) Fill in the blanks in the table below for the given sequence of loads; loads are performed sequentially from top to bottom. For each load in the table, specify the **tags** in the cache **after** the access and whether the load is a **hit/miss**. The first two rows are provided.

			Tags in the cache after the load				
			Set 0		Set 1		
Address (in hex)	Tag (in binary)	Set Index	Tag	Tag	Tag	Tag	H/M
0xC2	110	0	110	001	111	010	M
0x34	001	1	110	001	111	001	M
0x14	000	1	110	001	000	001	M
0xA0	101	0	110	101	000	001	M
0xC3	110	0	110	101	000	001	H
0x70	011	1	110	101	000	011	M
0xC2	110	0	110	101	000	011	H
0x11	000	1	110	101	000	011	H
0xAA	101	0	110	101	000	011	H
0x6F	011	0	011	101	000	011	M
0xC5	110	0	011	110	000	011	M
0x3D	001	1	011	110	000	001	M

- b) Assume that the 12 loads above are the very first memory accesses in the program. How many of the above cache misses (including the provided rows) are **cold misses**?

Answer:

6 cold misses (all misses except for 0xC5 and 0x3D)

2. Consider the following MIPS assembly program:

```
ori $t8, $zero, 0x2000
lw $t4, 0xC($t8)
lw $t5, 0x28($t8)
lw $t6, 0x30($t8)
lw $t7, 0x34($t8)
add $s0, $t4, $t5
add $s1, $t6, $t7
lw $s2, 0x4C($t8)
lw $s3, 0x10($t8)
```

The contents of some addresses in **data memory** are shown below. Recall that for MIPS, memory addresses are 32-bit, and memory is byte-addressable.

Memory Address (in hex)	Word Contents (in decimal)
0x00002000	-454
0x00002008	9
0x00002010	2
0x00002018	46
0x00002020	100
0x00002028	54
0x00002030	92
0x00002038	-434
0x00002040	-4
0x00002048	1
0x00002050	75
0x00002058	2

Memory Address (in hex)	Word Contents (in decimal)
0x00002004	77
0x0000200C	53
0x00002014	5899
0x0000201C	10
0x00002024	-44
0x0000202C	-78
0x00002034	20
0x0000203C	1111
0x00002044	7
0x0000204C	-9
0x00002054	3
0x0000205C	-410

- a) What are the contents of registers \$s0, \$s1, \$s2 and \$s3 **after** executing the program above?

Answer:

$$[\$s0] = 53 + 54 = 107$$

$$[\$s1] = 92 + 20 = 112$$

$$[\$s2] = -9$$

$$[\$s3] = 2$$

- b) You implement a 32B **direct-mapped** cache for data memory, with 8-byte cache blocks. Using the standard cache indexing scheme, the memory address is decomposed as:

memory address bits (standard)		
31 : 5	4 : 3	2 : 0
tag	index	offset

Unfortunately, it turns out you have a design **bug** in your cache where you only end up storing **26 bits per tag**:

memory address bits (buggy)		
31 : 6		4 : 3 2 : 0
tag		index offset

Fill in the table below, specifying for each load in the above program: the **(buggy) tag**, **set index**, **byte offset** in block, **hit/miss** and the **data word values** in the cache **after** the load. Assume the cache is initially empty. The first load is filled in for you.

					Cache Contents (Data Words) After Load			
Address (hex)	Tag (hex)	Index	Offset	H/M	Set 0	Set 1	Set 2	Set 3
0x0000200C	0x0000080	1	4	M	----	9	----	----
					----	53	----	----
0x00002028	0x0000080	1	0	H	----	9	----	----
					----	53	----	----
0x00002030	0x0000080	2	0	M	----	9	92	----
					----	53	20	----
0x00002034	0x0000080	2	4	H	----	9	92	----
					----	53	20	----

0x0000204C	0x0000081	1	4	M	----	1	92	----
					----	-9	20	----
0x00002010	0x0000080	2	0	H	----	1	92	----
					----	-9	20	----

- c) What are the contents of registers \$s0, \$s1, \$s2 and \$s3 **after** executing the program above with your buggy cache?

Answer:

[\$s0] = 53 + 9 = 62

[\$s1] = 92 + 20 = 112

[\$s2] = -9

[\$s3] = 92